

Confronto tra AC3 e AC4 in CSP (Sudoku)

Alberto Rovai - Studente della Università degli Studi di Firenze UNIFI

1. Introduzione

Lo scopo di questo lavoro è l'implementazione degli algoritmi di consistenza di arco AC3 e AC4 e il loro confronto sperimentale su alcune istanze di Sudoku. Le istanze di Sudoku sono state prelevate da siti di gioco online ([sudoku.com](https://www.sudoku.com)) e rappresentate manualmente nel file `/sudokus/sudokus_to_solve.py`.

Il confronto viene effettuato in termini di:

- Tempo di esecuzione
- Operazioni interne (revisioni / aggiornamenti)
- Valori rimossi dai domini
- Occupazione di memoria (dimensione delle strutture dati)

2. Modellazione del Sudoku come CSP

Il Sudoku deve essere rappresentato come un problema di soddisfacimento di vincoli (CSP):

1. Variabili: una per ogni cella, indicata con la chiave riga - colonna $((r,c))$
2. Dominio: valori $\{1,...,9\}$ oppure un singleton se la cella è già assegnata, vedi `costruisci_domini()`
3. Vincoli: due celle "vicine" devono avere valori diversi; per vicine si intendono celle nella:
 - stessa riga
 - stessa colonna
 - stesso blocco 3x3

Siccome i vincoli del sudoku tra celle vicine sono sempre gli stessi, qualsiasi sia la griglia da risolvere, i vicini li ricalcolo una sola volta e indipendentemente dai sudoku iniziali.

3. AC3

L'algoritmo AC3 rende un CSP arco-consistente sfruttando una coda di archi $((X_i, X_j))$, dove X_j è una cella "vicina". Iterativamente ogni arco della coda viene revisionato verificando se ogni valore del dominio di X_i ha almeno un supporto nel dominio di X_j , nel nostro caso se esiste almeno un valore nel dominio di X_j che sia diverso da quello nel dominio di X_i .

Se un valore viene eliminato, tutti gli archi entranti su (X_i) vengono reinseriti nella coda, eccetto quello appena inferito, ciò implica che possono essere reinseriti archi dove è già stata effettuata una revisione.

Le caratteristiche principali sono:

- Possibile revisione ripetuta degli stessi archi
- Uso limitato di memoria
- Buone prestazioni pratiche su Sudoku

I parametri che misuro sono:

- Tempo di esecuzione (cruciale per confronto computazionale)
- numero di revisioni di archi (chiamate a `revise()`)
- valori rimossi dai domini
- numero totale di archi (costante in sudoku)
- picco massimo della coda (memoria)

4. AC4

L'algoritmo AC4 migliora AC3 evitando la revisione ripetuta degli archi. Durante la fase di inizializzazione:

- per ogni tripla ((X_i, x, X_j)) viene calcolato il numero di supporti
- viene costruita una struttura inversa (`supportati_da`) che indica quali valori dipendono da un dato supporto

Quando un valore viene eliminato, vengono aggiornati solo i contatori effettivamente dipendenti da esso.

Le caratteristiche principali sono:

- Ogni arco viene considerato una sola volta
- Molto più lavoro nella fase di inizializzazione
- Maggiore uso di memoria
- Più operazioni elementari di bookkeeping

I parametri che misuro sono:

- Tempo di esecuzione (cruciale per confronto computazionale)
- numero di aggiornamenti dei contatori di supporto (non confrontabili direttamente con AC3 perché più elementare di `revise()`)
- valori rimossi dai domini
- numero di contatori (memoria usata)
- numero di strutture di supporto (memoria usata)
- picco massimo della coda dei valori da eliminare

5. Risultati sperimentali

I test sono stati effettuati su 5 istanze di Sudoku (`/sudokus/sudokus_to_solve.py`). I risultati sono riportati nella seguente tabella.

Sudoku	Algoritmo	Tempo (ms)	Operazioni	Valori rimossi	Strutture memoria	Picco coda	Celle risolte
1	AC3	22.66	9372 revisioni	408	1620 archi	6025	81/81
1	AC4	48.66	42822 aggiornamenti	408	9780 contatori / 489 supporti	394	81/81
2	AC3	17.36	8460 revisioni	360	1620 archi	6576	81/81
2	AC4	40.89	32634 aggiornamenti	360	8820 contatori / 441 supporti	438	81/81
3	AC3	28.78	6085 revisioni	235	1620 archi	4491	16/81

Sudoku	Algoritmo	Tempo (ms)	Operazioni	Valori rimossi	Strutture memoria	Picco coda	Celle risolte
3	AC4	68.49	30119 aggiornamenti	235	12020 contatori / 601 supporti	264	16/81
4	AC3	92.34	7415 revisioni	305	1620 archi	5589	31/81
4	AC4	60.59	33340 aggiornamenti	305	10100 contatori / 505 supporti	368	31/81
5	AC3	22.93	7111 revisioni	289	1620 archi	5499	26/81
5	AC4	52.02	31686 aggiornamenti	289	10420 contatori / 521 supporti	372	26/81

6. Analisi dei risultati

Il numero di operazioni riportato per AC3 e AC4 non è **direttamente confrontabile**:

- in AC3 una revisione corrisponde a una chiamata alla procedura `revise()` su un arco, dove confronta ogni valore del dominio della cella corrente
- in AC4 viene contato ogni aggiornamento dei contatori di supporto, un'operazione di livello più basso

AC4 **evita la revisione ripetuta** degli archi ma introduce un **numero maggiore di operazioni locali di aggiornamento e un maggiore consumo di memoria**. Entrambi gli algoritmi rimuovono **esattamente lo stesso numero di valori** dai domini, producendo inferenze equivalenti.

AC3 risulta **più veloce su Sudoku** di AC4 e occupa meno memoria. Oltretutto AC3 necessita soltanto di preparare gli archi per il confronto mentre AC4 prepara le strutture di supporto e i conteggi, questo comporta anche un tempo maggiore nel caso di un CSP contenuto come il Sudoku.

7. Conclusioni

- AC3 è più semplice, più leggero in memoria e più veloce su istanze di Sudoku
- AC4 riduce il lavoro ridondante sugli archi ma paga un costo elevato in inizializzazione e memoria
- I risultati sperimentali sono coerenti con la teoria

8. Riferimenti

- Russell, S., Norvig, P. "Artificial Intelligence: A Modern Approach", 4th ed., 2021
- Bessière, C. "Constraint Propagation", 2006